Vision & OCR Capability Specification

Project NOVA Screen Observation, Character Recognition, and Visual Element Grounding

---

Field	Value
Document ID	NOVA-SPEC-007
Version	1.0
Status	`APPROVED`
Author	Antigravity (Lead Software Engineering Agent)
Reviewer	ChatGPT (Chief Architect)
Approved By	Praveen (Project Founder)
Created	2026-06-28
Last Updated	2026-06-28
Dependencies	NOVA-SPEC-001, NOVA-SPEC-002, NOVA-SPEC-003, NOVA-SPEC-005

Revision History

Version	Date	Author	Summary of Changes
1.0	2026-06-28	Antigravity	Consolidate Vision Overview, Screen Capture, OCR, UI Detection, Grounding, Providers, and Safety specifications.

Table of Contents

Purpose & Scope

This specification defines the functional boundaries, API contracts, and security policies for the **Vision & OCR Capability**. It permits NOVA to capture screen pixels, perform character recognition, identify UI controls, and ground semantic planner targets to coordinates.

Constraint: The Vision capability is read-only. It parses screenshots but does not execute clicks or keystrokes directly.

Screen Capture Integration

- **Capture Sources:** Obtains display images from the `DesktopAutomationCapability` (full screen, active window, region, or multi-display configurations).
- **Buffer Pipeline:** Forwards captured images to OCR and element detection processors.

Optical Character Recognition (OCR)

- **Text Extraction:** Reads character sequences from screenshots.
- **Layout Preservation:** Maintains relative coordinate bounds (`bounding boxes`) for text blocks.
- **Confidence Metrics:** Yields probability scores for extracted words.
- **Language Profiles:** Supports multi-language dictionaries.

UI Element & Window Understanding

Processes screen displays to output structured control trees:

- **Element Classes:** Identifies buttons, menus, inputs, tables, icons, and popup alerts.
- **Metadata Bounds:** Outputs layout bounding boxes for detected targets.

Semantic Visual Grounding

Converts unstructured planner statements into target coordinates:

- *Input:* Semantic target descriptions (e.g. *"Click Chrome icon"*, *"Select profile Ravi"*).
- *Process:* Grounding model maps labels to active screen coordinates.
- *Output:* Virtual screen space coordinates (x, y) passed back to the execution engine.

Provider Abstraction Interfaces

To ensure model independence, the capability isolates layout engines behind two interface classes:

```
class IOCRProvider(ABC):
    @abstractmethod
    def extract_text(self, image_bytes: bytes) -> list[dict]: ...
    # Yields [{"text": "File", "bbox": (x, y, w, h), "confidence": 0.98}]

class IVisionModelProvider(ABC):
    @abstractmethod
    def detect_elements(self, image_bytes: bytes, categories: list[str]) ->
list[dict]: ...
    # Yields [{"label": "button", "center": (x, y), "bbox": (x1, y1, x2, y2)}]
```

Examples: PaddleOCR, Tesseract, EasyOCR, or API vision models implement these providers.

Observability & Privacy Policies

9. **In-Memory Buffer Rule:** Capture streams must reside only in-memory (e.g. `BytesIO`) and never write to the disk unless diagnostic logging is explicitly configured.
10. **Sensitive Information Masking:** OCR output labels matching patterns (like social security numbers or credit cards) must mask coordinates in memory logs.
11. **Visual Layout Caching:** Hashing algorithms verify if the screen is unchanged before running expensive OCR loops.

Testing & Accuracy Targets

- **OCR Precision:** OCR text extraction accuracy must exceed 95% on standard OS desktop system fonts.

- **Scaling Resilience:** Grounding tests check targets align correctly under high-DPI scaling offsets.
- **Theme Checks:** Elements must remain identifiable under light and dark UI themes.